

Intelligent Logistics Handling Robot

Technical Documentation

National University Student Engineering Innovation Competition

Harbin Institute of Technology (Shenzhen) | School of Robotics and Intelligent Manufacturing

Microcontroller: STM32F407 | SBC: Raspberry Pi 5

1. System Architecture

The robot adopts a distributed dual-processor architecture. The STM32F407 microcontroller is responsible for real-time motion control, motor driving, and sensor data acquisition. The Raspberry Pi 5 handles high-level tasks including vision processing, task scheduling, and decision-making. Inter-processor communication is achieved via UART serial protocol.



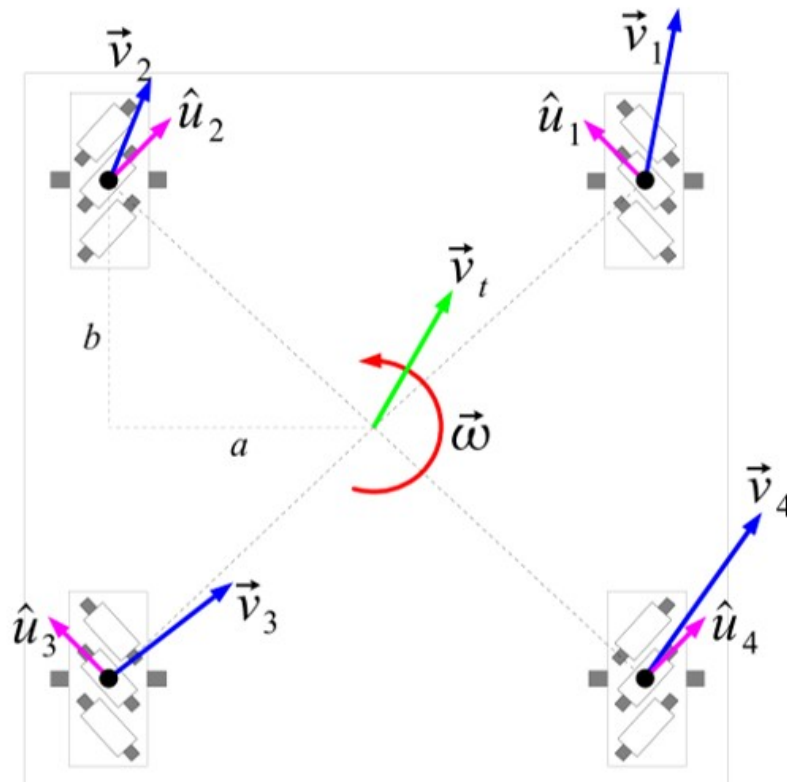
Figure 1: System Architecture Overview (STM32F407 + Raspberry Pi 5).

2. Core Algorithms

2.1 Motion Control

Mecanum Inverse Kinematics

To achieve precise omnidirectional motion, an inverse kinematics model is established in the robot body coordinate frame. The model maps desired linear velocities (v_x , v_y) and angular velocity (ω) to individual wheel speeds, enabling full-pose omnidirectional movement including simultaneous rotation and translation.



$$\begin{cases} v_{w1} = v_{ty} - v_{tx}\omega(ab)) \\ v_{w2} = v_{ty}v_{tx} - \omega(ab)) \\ v_{w3} = v_{ty} - v_{tx} - \omega(ab)) \\ v_{w4} = v_{ty}v_{tx}\omega(ab)) \end{cases}$$

Control Strategy & Filtering

An incremental PID position controller is implemented, using encoder feedback pulses as the control variable. Several enhancements are integrated:

- Conditional Integration: Prevents integral windup by freezing the integral term when the actuator saturates.
- Safety Limits: Hard constraints on PWM output to protect hardware.
- 4th-Order Butterworth Filter: Applied to gyroscope angular velocity data to suppress high-frequency noise.

A second-order Butterworth low-pass filter with a cutoff frequency of 20 Hz is specifically designed for the gyroscope signal chain, effectively mitigating measurement noise that would otherwise excite the PID controller.

2.2 Vision Algorithm Pipeline

A multi-stage image processing pipeline is designed to achieve sub-pixel-level localization of material centers under varying lighting conditions.

HSV Thresholding

The image is converted from RGB to HSV color space. Dynamic threshold intervals are set based on the target color ring's hue channel, effectively overcoming environmental illumination variations.

Morphological Operations

Erosion followed by dilation is applied to eliminate isolated noise pixels and fill holes within target regions, improving contour integrity.

Hough Circle Transform

An improved Hough gradient method is applied to detect geometric contours of target objects. Candidate circle centers within the expected radius range are filtered and ranked.

Sub-pixel Center Extraction

Based on coarse localization, the Moments (grayscale centroid) method is used to refine circle center coordinates to sub-pixel accuracy. Localization error is controlled within ± 1 mm, satisfying the high-precision grasping requirement of the robotic arm.

3. Key Performance Metrics

Performance Metric	Specification	Description
Positioning Accuracy	± 2 mm	Within 1 m range
Response Time	<100 ms	Command-to-execution latency
Max Speed	0.5 m/s	Safe operating velocity
Recognition Range	0.3–1.5 m	Valid object detection range
Grasping Precision	± 1 mm	Robotic arm end-effector accuracy
Comm. Latency	<50 ms	STM32–Raspberry Pi UART delay

4. Issues & Resolutions

4.1 Motor Position Anomaly (8-Digit Distance Values)

Symptom: distance_traveled of Wheel 1 and Wheel 3 displayed 8-digit values.

Root Cause: Encoder data overflow or communication parsing error.

Resolution:

1. Checked motor wiring for loose connections.
2. Re-calibrated encoder zero position.
3. Verified the data parsing logic in the ParseMotor_Status function.
4. Switched from absolute position to relative position calculation.

4.2 Gyroscope Data Jump

Symptom: Angle suddenly jumped by $\pm 360^\circ$.

Resolution: Added angle continuity checking in the angle processing routine. Refer to the angle continuity handling code in main.c.

4.3 PID Oscillation

Symptom: Vehicle oscillated during movement.

Resolution:

5. Reduced PID gains, especially the D term.
6. Increased filtering coefficient.
7. Inspected mechanical mounting for rigidity.
8. Adjusted control period.

5. Code Snippets

5.1 World Frame ↔ Body Frame Velocity Transformation

Implementation of full-pose omnidirectional motion (Mecanum inverse kinematics):

```
void Publish_motor_speed(void)
{
    float angle = omega * pi / 180.0f; // 角度转弧度
    // 根据map_speed计算car_speed
    car->target_car_Vx = car->target_map_Vx * cosf(angle) + car->target_map_Vy * sinf(angle);
    car->target_car_Vy = -car->target_map_Vx * sinf(angle) + car->target_map_Vy * cosf(angle);
    // 根据car_speed计算motor_speed
    float Car_H = 1.670f; // 前后轮心距
    float Car_W = 2.380f; // 左右轮心距
    float Kx = 1;
    float Ky = 1;
    float Kz = 1.74 * 1; // 1.74将target和current单位统一

    car->target_Vel1 = Kx * car->target_car_Vx - Ky * car->target_car_Vy + Kz * (-car->target_angle_speed * (Car_H / 2 + Car_W / 2));
    car->target_Vel2 = Kx * car->target_car_Vx + Ky * car->target_car_Vy + Kz * (car->target_angle_speed * (Car_H / 2 + Car_W / 2));
    car->target_Vel3 = Kx * car->target_car_Vx + Ky * car->target_car_Vy + Kz * (-car->target_angle_speed * (Car_H / 2 + Car_W / 2));
    car->target_Vel4 = Kx * car->target_car_Vx - Ky * car->target_car_Vy + Kz * (car->target_angle_speed * (Car_H / 2 + Car_W / 2));

    // 线性平滑
    car->target_Vel1 = car->target_Vel1 * 0.85 + car->current_Vel1 * 0.15;
    car->target_Vel2 = car->target_Vel2 * 0.85 + car->current_Vel2 * 0.15;
    car->target_Vel3 = car->target_Vel3 * 0.85 + car->current_Vel3 * 0.15;
    car->target_Vel4 = car->target_Vel4 * 0.85 + car->current_Vel4 * 0.15;

    uint8_t acc = 150;
    bool snF = true;
    uint32_t delay_ms = 5;
    Emm_V5_Send_Four_Vel_Control(car->target_Vel1, car->target_Vel2, car->target_Vel3, car->target_Vel4, acc, snF, delay_ms);
}
```

5.2 PID Control Loop

Adaptive PID tuning for fast and precise positioning:

```
float PID_calc_micro(float error_position, PIDController *PID)
{
    if (PID->first_call == true)
    {
        PID->first_call = false;
        PID->last_call_time = HAL_GetTick();
        return 0;
    }

    uint32_t current_time = HAL_GetTick();
    float actual_dt = (current_time - PID->last_call_time) / 1000.0f;
    actual_dt = fmaxf(fminf(actual_dt, 0.1f), 0.005f);
    // 这个用于限制dt的，因为dt不能太大，也不能太小，所以需要限制
    // 这个是哪里来的莫名其妙的代码，我们的时间很长，不要被莫名其妙的限制幅度了

    PID->last_call_time = current_time;
    PID->dt = actual_dt;

    float error = error_position;
    float abs_error = fabsf(error);

    //===== 四阶巴特沃斯滤波（调整为8Hz截止频率） =====//

    float raw_deriv = (error - PID->prev_error) / actual_dt;
    float filtered_deriv = 0.032f * raw_deriv + 0.198f * PID->deriv_buf[0] + 0.468f * PID->deriv_buf[1] + 0.198f * PID->deriv_buf[2] + 0.032f * PID->deriv_buf[3];
    memmove(&PID->deriv_buf[1], PID->deriv_buf, 3 * sizeof(float));
    PID->deriv_buf[0] = filtered_deriv;

    //===== 输出合成（使用已声明的comp_term） =====//
```

```

//===== 条件积分模块 =====//
if (fabs(error) < 2.0f) // 直接按照积分模块去那啥咯
{ // 仅在误差<目标值的5%时激活
    PID->integral += error * PID->dt * PID->ki; // Ki=0.3

    // 积分限幅（限制在最大输出的10%）

    PID->integral = fmaxf(fminf(PID->integral, PID->max_output), -PID->max_output);
}
else
{
    PID->integral *= 0.5f; // 快速释放历史积分
}
// 对积分项目进行了修改，integral现在是已经加上了积分增益的

PID->integral += error * PID->dt * PID->ki;

if (error * PID->prev_error < 0.0f)
{
    PID->integral = 0.0f; // 快速释放历史积分
}

// 合成输出（新增积分项）
output += PID->integral;

//===== 增强型安全限幅 =====//

output = fmaxf(fminf(output, PID->max_output), -PID->max_output);
// output = 0.6f * output + 0.4f * PID->last_output; // 调整惯性系数

PID->last_output = output;

//===== 末端微调防抖 =====//
// if (abs_error < 5.0f)
// {
//     output *= 0.3f; // 极近端输出衰减
// }

PID->prev_error = error;
return output;
}

```

5.3 Vision Processing Functions

The `get_center` function integrates the following pipeline stages:

- LAB + CLAHE illumination enhancement
- HSV color segmentation
- Morphological operations (erosion & dilation)
- Contour filtering (area + aspect ratio)
- Multi-frame statistics + stillness detection


```

lab = cv2.cvtColor(frame, cv2.COLOR_BGR2LAB)
l, a, b = cv2.split(lab)
clahe = cv2.createCLAHE(cliplimit=3.0, tileGridSize=(8,8))
c1 = clahe.apply(l)
enhanced = cv2.merge((c1,a,b))
frame = cv2.cvtColor(enhanced, cv2.COLOR_LAB2BGR)
# frame = cv2.medianBlur(frame, 3)
hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)

# 创建颜色掩膜并处理红色物块
red_mask1 = cv2.inRange(hsv, np.array(dim_red_min), np.array(dim_red_max))
red_mask2 = cv2.inRange(hsv, np.array(dim_red_min1), np.array(dim_red_max1))
red_mask = cv2.bitwise_or(red_mask1, red_mask2)

# 应用高斯滤波
red_blurred = cv2.GaussianBlur(red_mask, (5, 5), 0)

# 应用形态学操作增强
red_kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))
red_opened = cv2.morphologyEx(red_blurred, cv2.MORPH_OPEN, red_kernel)
red_closed = cv2.morphologyEx(red_opened, cv2.MORPH_CLOSE, red_kernel)
# 查找红色物块轮廓
red_contours, _ = cv2.findContours(red_closed, method=cv2.RETR_TREE, mode=cv2.CHAIN_APPROX_NONE)
for cnt in red_contours:
    area = cv2.contourArea(cnt)
    # print("red_area:", area)
    # 计算轮廓的周长
    if (MIN_AREA < area < MAX_AREA): # 过滤小和大的轮廓, 并确保轮廓接近圆形
        x, y, w, h = cv2.boundingRect(cnt)
        if (1.4 > (w/h) > 0.7):
            x_center = (x + w / 2) / 640
            y_center = (y + h / 2) / 480
            color_coords_dict["red"].append((x_center, y_center))

    # 绘制轮廓而不是矩形框
    cv2.drawContours(frame, [cnt], 0, (0, 0, 255), 2)

```

Hough Circle detection optimization strategy:

- Two-stage Hough hierarchical detection
- Circle center matching and grouping
- Median filtering for outlier rejection
- Temporal-domain exponential smoothing

```

def get_ring_center(self, frame, r_min=70, r_max=100, smooth_factor=0.4):
    """
    获取圆环的中心坐标

    参数:
    frame : 输入的图像帧
    r_min (int): 圆环内径的最小半径
    r_max (int): 圆环外径的最大半径

    返回:
    圆环的中心坐标 (x, y, r)
    """
    median = cv2.medianBlur(frame, 3)
    grayImg = cv2.cvtColor(median, cv2.COLOR_BGR2GRAY)
    # cv2.imshow("grayImg", grayImg)
    grayImg = cv2.GaussianBlur(grayImg, (5, 5), 0)
    cannyImg = cv2.Canny(grayImg, 50, 150)
    # cv2.imshow("cannyImg", cannyImg)
    circle_size = 0
    pre_list = []
    circles_list = []
    res_list = []

    circles_pre = cv2.HoughCircles(grayImg, cv2.HOUGH_GRADIENT_ALT, 1.5, 80,
                                   param1=240, param2=0.80,
                                   minRadius=r_min, maxRadius=r_max)

    # 检查第一次检测结果
    if circles_pre is None:
        # print("未检测到初始圆环")
        return []

    circles_pre = np.round(circles_pre[0, :]).astype("int")
    pre_list = [(x, y, r) for (x, y, r) in circles_pre]
    circle_size = len(pre_list)

```

```

# 第二次检测
circles = cv2.HoughCircles(grayImg, cv2.HOUGH_GRADIENT_ALT, 1.8, 30,
                             param1=340, param2=0.95,
                             minRadius=r_min, maxRadius=r_max)
if circles is None:
    # print("未检测到精细圆环")
    return []

circles = np.round(circles[0, :]).astype("int")

# 初始化分类列表
circles_list = [[] for _ in range(circle_size)]

# 匹配逻辑
for (x, y, r) in circles:
    if circle_size > 0:
        distances = [(i, (x-pre_x)**2 + (y-pre_y)**2) for i, (pre_x, pre_y, _) in enumerate(pre_list)]
        if distances:
            closest_idx, closest_dist = min(distances, key=lambda item: item[1])
            # 只有当距离小于阈值时才匹配
            if closest_dist < (r * 0.5)**2: # 距离阈值为半径的一半
                circles_list[closest_idx].append((x, y, r))
            else:
                # 距离太远, 可能是新的圆, 创建新分组
                if len(circles_list) < 4: # 限制最大分组数
                    circles_list.append([(x, y, r)])
                    circle_size += 1

```

```

# 滤波得到精细坐标
for i in range(circle_size):
    x_sum = 0
    y_sum = 0
    r_sum = 0 # 添加半径求和
    size = len(circles_list[i])

    for x, y, r in circles_list[i]:
        # cv2.circle(frame, (x, y), r, (0, 255, 0), 1)
        x_sum += x
        y_sum += y
        r_sum += r # 累加半径

    if(size != 0):
        # 使用中值滤波而非平均值
        # 在计算圆心时
        if size >= 3: # 至少需要3个点才能计算中值
            # 分别提取x,y,r坐标列表
            x_list = [x for x,_,_ in circles_list[i]]
            y_list = [y for _,y,_ in circles_list[i]]
            r_list = [r for _,_,r in circles_list[i]]

            # 计算中值
            x_med = sorted(x_list)[len(x_list)//2]
            y_med = sorted(y_list)[len(y_list)//2]
            r_med = sorted(r_list)[len(r_list)//2]

            res_list.append((x_med, y_med, r_med))
        else:
            # 点太少, 使用平均值
            x_avg = x_sum / size
            y_avg = y_sum / size
            r_avg = r_sum / size
            res_list.append((x_avg, y_avg, r_avg))

```

```
# 在返回结果前应用时间滤波
res_list = self.apply_temporal_filter(res_list,smooth_factor)

# 修改顶部坐标显示，增加半径信息
if len(res_list) > 0:
    coords_text = " "
    for idx, (x, y, r) in enumerate(res_list): # 注意这里增加了r
        coords_text += f"({x:.2f},{y:.2f},R={r:.2f})"
        if idx < len(res_list) - 1:
            coords_text += ", "

    cv2.putText(frame, coords_text, (10, 30), cv2.FONT_HERSHEY_SIMPLEX,
                0.7, (0, 255, 255), 2)
return res_list
```

6. Source Code & References

Project Repository (GitLab):

<https://gitlab.com/Collapsar-ss/logistics-handling.git>

A GitHub mirror is maintained for accessibility. The repository includes full source code for STM32 firmware (C), Raspberry Pi vision modules (Python), communication protocol definitions, and a comprehensive README documenting the build process and hardware setup.

Keywords: STM32F407, Raspberry Pi 5, Mecanum Wheels, Inverse Kinematics, PID Control, Butterworth Filter, OpenCV, HSV Thresholding, Hough Circle Transform, Sub-pixel Localization, UART Communication, Omnidirectional Mobile Robot.